

A Multi-Label Text Classification Approach to Tweet Toxicity and Severity Using Encoder Only Transformer

Haluk Harun Gündoğan

June 20, 2024

Abstract

X, formerly known as Twitter, is a platform that allows users to freely share their opinions. However, tweets that violate community guidelines are subject to removal or the banning of that particular user. This paper presents a multi-label text classification approach for assessing tweet toxicity and severity using various neural network architectures. The study investigates the performance of several models, including Long Short-Term Memory (LSTM), Convolutional 1D-Bidirectional Long Short-Term Memory (CONV1D-BiLSTM), an enhanced CONV1D-BiLSTM with Multi-Head Attention Mechanism, and both Encoder-only and traditional Transformer models. Experiments on a comprehensive dataset of tweets reveal that the Encoder-only Transformer model outperforms others in terms of F1 and computational efficiency with a Micro F1 score of 67%. This research highlights the effectiveness of advanced Transformer architectures in handling the complexities of multi-label text classification tasks.

1 Introduction

As articulated in the abstract, the primary objective at hand entails the classification of tweets based on the severity of toxicity inherent within the content. In order to achieve this goal, the initial step necessitates the identification of whether a given tweet exhibits toxic characteristics, which necessitates a binary-classification model. Following this determination, the subsequent phase involves multi-label classification, wherein tweets are categorized into distinct classes, namely "severe toxic", "obscene", "threat", "insult", and "identity hate". It is imperative that the adopted model demonstrates proficiency in discerning if a tweet aligns with multiple categories, as decisions and interventions will be predicated upon the assigned labels. [KHA20]

The rise of social media platforms has facilitated the widespread sharing of information and ideas, but it has also led to an increase in toxic and harmful content. Identifying and mitigating such content is crucial for maintaining a safe and respectful online environment. Traditional text classification methods for tweet classification, particularly on this dataset have shown some discrepancies by the way that they handle the problem description as well as by the way that they evaluate the result. For instance using accuracy as the evaluation metric for a long-tailed dataset would inevitably lead to misinterpretation of the results. [HGK⁺21]

Moreover, the loss function should also be adjusted accordingly, in order to handle all the class distribution as fairly as possible to provide convergence. [HGK⁺21]

This paper is structured as follows: the methodology section details the data preprocessing steps, model architectures, and training procedures. The results section presents the comparative performance of the models, followed by a discussion of the findings and their implications for real-world applications. Finally, the conclusion highlights the key contributions and suggests directions for future research.

2 Design of the Data

The design of the dataset as well as the chosen model architectures have been handled in the following steps

2.1 Data

The [dataset](#) comprises eight columns with over 150.000 rows, including "id," "comment_text," "toxic," (15294) "severe_toxic," (1595) "obscene," (8449) "threat," (478) "insult," (7877) and "identity_hate." (1405) where the values in parenthesis specify the number of instances of that particular class in the dataset. Each row represents a unique instance within the dataset, with the "id" column serving as a unique identifier for individual comments. The "comment_text" column contains the textual content of the comments posted. Additionally, the dataset features binary indicators for toxicity-related categories, including "toxic," "severe_toxic," "obscene," "threat," "insult," and "identity_hate," each denoting 0 or 1 depending on whether the corresponding comment exhibits toxicity in the specified category.

2.2 Preprocessing

In the preprocessing phase, special operations like concatenation within the tweet texts were deemed inappropriate for the purpose of this paper. To remedy this, the Regular Expressions were utilized to eliminate these operations, retaining only the tweets themselves. Additionally, since the tweets were treated as sentences, further preprocessing was conducted. This involved removing '/' characters and separating subfields with spaces to uphold a sentence format. These transformations, implemented via Regular Expressions, ensured uniformity and suitability for subsequent stages of model training. Normalization techniques then were employed to standardize the textual data, such as converting all characters to lowercase and removing punctuation marks, thereby ensuring consistency in text representation and reducing redundancy.

2.3 Tokenization

Subsequent to data cleaning and preprocessing, tokenization is applied to segment the textual data into individual tokens, such as words or phrases. This step facilitates subsequent text processing tasks by breaking down the comments into manageable units for analysis. This phase also have different tokenization based on the model at hand. Subsequent to tokenization, the vocabulary for the dataset has been created.

2.4 Word Embeddings

In preparation for modeling, the textual data is transformed into numerical representations through embedding techniques, This conversion facilitates the model learning which can be only performed on the numerical data. In order to get these embeddings GloVe Embeddings were used, which have been trained on a similar dataset involving tweets.

3 Design of the Models

This section delves into the comparison between traditional Long-Short Term Memory (LSTM) models which are successors of traditional Recurrent Neural Networks(RNN) , and modern Transformer models, highlighting their respective strengths and weaknesses in natural language processing (NLP) tasks. While RNNs excel in tasks like language translation due to their sequential processing nature and contextual understanding, they struggle with long-range dependencies. Although the LSTM architecture helps with retaining these long term dependencies, problems like gradient decay are inevitable. In contrast, Transformers, equipped with self-attention mechanisms, handle long-range dependencies by processing sequences in parallel. The integration of these models in encoder decoder architectures is pivotal for advancing sequence modeling and enhancing language comprehension. By exploring various encoder-decoder structures and sequence-to-sequence models, the aim is to optimize performance across diverse NLP tasks, leveraging the strengths of different architectures and refining attention mechanisms to improve precision and coherence in generating output sequences.[AD22]

3.1 LSTM

First model, the LSTM model, starts with an input layer that accepts sequences of a fixed length. It then uses an embedding layer to convert these sequences of word indices into dense vectors of size using a pre-trained embedding matrix, which is not trainable to maintain the semantic richness of the pre-trained embeddings. The output of the embedding layer is passed to an LSTM layer with 128 units, which is designed to capture temporal dependencies in the data while mitigating overfitting through dropout and recurrent dropout rates of 0.1. The LSTM layer outputs a sequence of hidden states, which are then summarized using a global average pooling layer to reduce the dimensionality and retain important features. The resulting pooled vector is fed into a dense layer with 128 units and ReLU activation to introduce non-linearity and further learn complex representations. A dropout layer follows to prevent overfitting by randomly setting 10% of the input units to zero during training. Finally, the model outputs predictions through a dense layer with six units and sigmoid activation, suitable for multi-label classification tasks, as each unit independently predicts the probability of a corresponding label.

3.2 Conv-BiLSTM

This model begins with an input layer designed to handle sequences of a specified maximum length. It then employs an embedding layer to transform these sequences of word indices into dense vectors of size embedding size, utilizing a pre-trained embedding matrix that remains fixed during training to retain its learned semantic properties. Following the embedding, a SpatialDropout1D layer with a dropout rate of 0.2 is applied to mitigate overfitting by randomly setting entire 1D feature maps to zero, enhancing the model's generalization. Next, a 1D convolutional layer with 64 filters and a kernel size of 3, initialized with the Glorot uniform method, captures local patterns in the sequence data. The output is then processed by a bidirectional LSTM layer with 128 units, which reads the sequence in both forward and backward directions to capture context from both ends, incorporating dropout and recurrent dropout rates of 0.1 to prevent overfitting. The sequence of LSTM outputs is summarized using global average pooling, reducing the dimensionality while retaining key features. This pooled output is then passed through a dense layer with 128 units and ReLU activation to introduce non-linearity and learn complex representations. Another dropout layer follows, setting 10% of the input units to zero during training to avoid overfitting. Finally, the model predicts six independent probabilities using a dense layer with sigmoid activation, suitable for multi-label classification tasks. [ABS⁺23]

3.3 Conv-BiLSTM with Multi-Head Attention

This model starts with an input layer that accepts sequences of a maximum length. It then uses an embedding layer to convert these sequences into dense vectors of embedding size, leveraging a pre-trained embedding matrix that remains fixed during training to preserve its semantic richness. Following this, a 1D Spatial Dropout layer with a dropout rate of 0.2 is applied to prevent overfitting by randomly dropping entire 1D feature maps. The model then uses a 1D Convolution layer with 64 filters and a kernel size of 3, initialized with the Glorot uniform method, to capture local patterns in the data. Next, a bidirectional LSTM layer with 128 units processes the sequence in both forward and backward directions, enhancing context capture, with dropout and recurrent dropout rates of 0.1 to further prevent overfitting. This is followed by a multi-head self-attention mechanism with 8 heads and embedding dimension equal to embedding size, which allows the model to focus on different parts of the sequence simultaneously, enhancing the learning of complex dependencies. The output is then summarized using global average pooling to reduce dimensionality while retaining important features. A dropout layer with a rate of 0.1 is applied next to avoid overfitting. Finally, the model outputs predictions through a dense layer with six units and sigmoid activation, making it suitable for multi-label classification by independently predicting the probability of each label. [LLWW23]

3.4 Traditional Transformer

This model employs a Transformer-based architecture, beginning with an input layer that handles sequences of a specified maximum length. The sequences are transformed into dense vectors of size embedding size using a pre-trained embedding matrix that remains fixed during training to preserve

its learned semantic properties. Positional encoding is added to these embeddings to inject information about the relative or absolute position of tokens in the sequence, crucial for the Transformer architecture which lacks inherent sequential information.

The encoder section of the model consists of multiple (in this case, four) Transformer encoder layers. Each layer incorporates multi-head self-attention mechanisms (with 8 heads) to allow the model to focus on different parts of the sequence simultaneously, followed by feed-forward layers (with 512 units) to learn complex representations. Layer normalization is applied to the output of the encoder to stabilize and accelerate training.

For the decoder, a separate input layer and embedding layer, also using the pre-trained embedding matrix, are used to process the target sequences. Positional encoding is again added to these embeddings. The decoder also consists of multiple Transformer decoder layers, which use the encoder output for context. Each decoder layer includes masked multi-head self-attention, encoder-decoder attention to integrate the encoder’s context, and feed-forward layers. The final decoder output is layer normalized, then pooled globally to reduce its dimensionality while retaining important features. A dropout layer with a rate of 0.1 is applied to prevent overfitting. The model concludes with a dense layer using sigmoid activation to independently predict the probability of each of six possible labels, making it suitable for multi-label classification tasks.

3.5 Encoder Only Transformer

This model employs once again a Transformer-based architecture designed for sequence classification tasks. The model starts with an input layer to handle sequences of a specified maximum length. These sequences are converted into dense vectors using an embedding layer with a fixed, pre-trained embedding matrix to retain the learned semantic properties of words.

The core of the model consists of multiple Transformer encoder layers, each featuring an embedding dimension matching the size of the embeddings, 8 attention heads, and a feed-forward network with 512 units. These encoder layers are repeated four times to effectively capture complex patterns in the input data. The multi-head self-attention mechanism within each encoder layer allows the model to focus on different parts of the sequence simultaneously, enhancing its ability to learn intricate dependencies. The feed-forward network processes the attention outputs to further refine the representations.

After passing through the Transformer encoder layers, layer normalization is applied to stabilize and improve the convergence during training. The normalized outputs are then globally averaged pooled, reducing the dimensionality and summarizing the sequence information into a fixed-size vector. A dropout layer with a 0.1 dropout rate is included to prevent overfitting by randomly setting 10% of the input units to zero during training. Finally, the model produces its predictions through a dense layer with six units and a sigmoid activation function. This setup is suitable for multi-label classification, where each unit independently predicts the probability of a corresponding label, allowing the model to handle tasks with multiple possible outputs.[ABS+23]

4 Implementation

4.1 Computational Environment Setup

The process of training and testing have been conducted on Google Colab, utilizing the T4 GPU runtime for accelerated computations. This cloud-based environment provided the necessary resources for efficient model training and evaluation, ensuring optimal performance and scalability throughout the experimentation phase.

4.2 Model Training

The model’s compilation involved specifying binary cross-entropy as the chosen loss function and employing the Adam optimizer with a fixed learning rate of $1e-3$. The evaluation during training focused on optimizing the micro F1 score, ensuring comprehensive performance assessment.

To gain insights into the model’s architecture and parameters, the summary has been shown in real-time while models were being trained, providing a detailed overview of its layers and configurations. Subsequently, the model was trained using the fit method. The training dataset consisted of

paired inputs and corresponding . Over the course of 10 epochs for LSTM models, and 15 epochs for Transformer models each comprising batches of 128 samples, the models iteratively adjusted their parameters to minimize the defined loss of binary cross entropy and enhance predictive accuracy. Progress updates were displayed, facilitating real-time monitoring of training dynamics.

The reason for the change in amount of epochs for LSTM models and Transformer based models is that, training an epoch of an LSTM model takes almost three times more than of a Transformer based model. That is why the hyperparameter adjusted accordingly in order to facilitate fairness.

Moreover, a validation split of 20% was implemented to validate the model’s performance on unseen data, ensuring robustness across different datasets and scenarios.

5 Experiments and Results

In the context of multilabel classification tasks, evaluating the performance of the model requires considering multiple aspects of its predictions across various labels or categories. Several evaluation metrics, precision, recall, F1-score, and micro F1-score offer valuable insights into the model’s effectiveness in handling multilabel data.

5.1 Evaluation Metrics

Precision measures the proportion of correctly predicted positive instances (true positives) out of all instances predicted as positive by the model. In multi-label classification, precision is calculated independently for each label.

Recall, also known as sensitivity, quantifies the proportion of correctly predicted positive instances out of all actual positive instances. Like precision, it is calculated independently for each label.

F1-score is the harmonic mean of precision and recall, providing a balanced measure of a model’s performance. It considers both false positives and false negatives and is particularly useful when dealing with imbalanced datasets.

Precision, recall, and F1-score offer insights into the model’s ability to correctly identify relevant labels while avoiding false positives and false negatives. Higher values indicate better performance in terms of precision, recall, and F1-score.

5.2 Precision

Precision is defined as:

$$\text{Precision} = \frac{TP}{TP + FP}$$

where TP is the number of true positives and FP is the number of false positives.

5.3 Recall

Recall is defined as:

$$\text{Recall} = \frac{TP}{TP + FN}$$

where FN is the number of false negatives.

5.4 F1 Score

The F1 score is the harmonic mean of precision and recall:

$$F1 = 2 \cdot \frac{\text{Precision} \cdot \text{Recall}}{\text{Precision} + \text{Recall}}$$

5.5 Micro F1 Score

In the context of multi-class classification, the micro F1 score is computed as:

$$\text{Micro F1} = \frac{2 \cdot \text{Micro Precision} \cdot \text{Micro Recall}}{\text{Micro Precision} + \text{Micro Recall}}$$

where Micro Precision and Micro Recall are computed across all classes:

$$\text{Micro Precision} = \frac{\sum_i TP_i}{\sum_i TP_i + \sum_i FP_i}$$

$$\text{Micro Recall} = \frac{\sum_i TP_i}{\sum_i TP_i + \sum_i FN_i}$$

5.6 Comparison of Model Performances

Models	Total Parameters	Micro F1 Score
LSTM	2.185.734	57%
Conv-BiLSTM	2.269.766	62%
Conv-BiLSTM with Multi Head Attention	3.881.606	58%
Traditional Transformer	21.074.102	62%
Encoder Only Transformer	13.932.102	67%

Table 1: Comparison of Total Parameter Count and Micro F1 Score for Different Models

The comparison of various neural network models reveals distinct trade-offs between parameter efficiency and performance, measured by Micro F1 Score. Among the models evaluated, LSTM and Conv-BiLSTM exhibit fewer parameters (2,185,734 and 2,269,766 respectively) but achieve lower Micro F1 Scores (57% and 62%). In contrast, models leveraging Transformer architectures show higher performance: the Traditional Transformer with 21,074,102 parameters achieves a Micro F1 Score of 62%, while the Encoder Only Transformer, with 13,932,102 parameters, achieves the highest Micro F1 Score of 67%. This suggests that while Transformers require more parameters, they deliver superior performance in tasks demanding high F1 Scores, such as classification or sequence modeling. Therefore, the choice between these models depends on balancing computational resources with the need for optimized predictive accuracy in specific applications.

6 Conclusion

Based on the results presented, the critical importance of advanced neural network architectures, particularly Transformer models, for effectively classifying tweets based on their toxicity levels is underscored by our study. The comparative analysis revealed that while efficiency in parameter usage is offered by LSTM and Conv-BiLSTM models, they are significantly outperformed by Transformer-based approaches in terms of Micro F1 Score, demonstrating their capability to discern nuanced distinctions in toxic content across multiple categories. This finding not only validates the efficacy of Transformer architectures in handling complex text classification tasks but also emphasizes the necessity of choosing models that prioritize performance metrics like F1 Score over simple accuracy, especially in datasets with imbalanced class distributions. Moving forward, enhancements in Transformer variants or novel techniques could be explored to further refine tweet classification accuracy and efficiency, thereby contributing to safer and more respectful interactions within online communities.

References

- [ABS⁺23] Iqra Ameer, Necva Bölücü, Muhammad Hammad Fahim Siddiqui, Burcu Can, Grigori Sidorov, and Alexander Gelbukh. Multi-label emotion classification in texts using transfer learning. *Expert Systems with Applications*, 213:118534, 2023.

- [AD22] Enrique Amigo and Agustín Delgado. Evaluating extreme hierarchical multi-label classification. In Smaranda Muresan, Preslav Nakov, and Aline Villavicencio, editors, *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pages 5809–5819, Dublin, Ireland, May 2022. Association for Computational Linguistics.
- [HGK⁺21] Yi Huang, Buse Giledereli, Abdullatif Köksal, Arzucan Özgür, and Elif Ozkirimli. Balancing methods for multi-label text classification with long-tailed class distribution. In Marie-Francine Moens, Xuanjing Huang, Lucia Specia, and Scott Wen-tau Yih, editors, *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*, pages 8153–8161, Online and Punta Cana, Dominican Republic, November 2021. Association for Computational Linguistics.
- [KHA20] Navid Khezrian, Jafar Habibi, and Issa Annamoradnejad. Tag recommendation for online qa communities based on bert pre-training technique, 2020.
- [LLWW23] Guangyao Lu, Yuling Liu, Jie Wang, and Hongping Wu. Cnn-bilstm-attention: A multi-label neural classifier for short texts with a small set of labels. *Information Processing Management*, 60(3):103320, 2023.